Real-Time Predictive Maintenance for EV Battery State of Health via Streaming Architecture and Agentic AI

Muhammad Hassan Siddiqui
Department of Computer Science
University of Central Punjab
Lahore, Pakistan

***Abstract*—The rapid proliferation of Electric Vehicles (EVs) necessitates robust Battery Management Systems (BMS) to prevent catastrophic failures and optimize the State of Health (SOH) [3]. Traditional predictive maintenance pipelines rely on batch-processing orchestrators, which introduce unacceptable latency for critical thermal or degradation events. This paper presents a highly scalable, real-time streaming architecture for EV battery SOH monitoring. By decoupling high-velocity telemetry ingestion from computationally heavy Large Language Model (LLM) workflows, we achieve millisecond-level inference latency. The system utilizes Apache Kafka and Redis streams for stateful data buffering, a dual LSTM/Temporal Convolutional Network (TCN) inference ensemble for continuous SOH prediction, and a LangGraph-orchestrated LLM agent for on-demand, actionable maintenance reporting. A per-battery chronological evaluation shows the LSTM baseline achieving a mean absolute error of 0.0151 Ah ($R^2 = 0.634$), outperforming the TCN's 0.0206 Ah ($R^2 = 0.550$) on this dataset, and both models are retained in an averaging ensemble for redundancy. Load testing demonstrates a 0% failure rate on the core inference endpoint under concurrent load, validating the architecture's viability for enterprise-scale EV fleets.**

***Index Terms*—Electric Vehicles, Predictive Maintenance, Apache Kafka, Temporal Convolutional Networks, Long Short-Term Memory, State of Health, MLOps, Generative AI.**

I. Introduction

The transition towards sustainable transportation has positioned Electric Vehicles (EVs) at the forefront of the automotive industry. However, the lithium-ion battery pack remains the most expensive and volatile component of an EV, often accounting for nearly half of the vehicle's total cost. Gradual capacity degradation reduces the effective driving range, while abrupt internal failures can lead to thermal runaway, a severe safety hazard. Consequently, transitioning from reactive maintenance to AI-driven predictive maintenance is a critical mandate for fleet operators and automotive manufacturers.

Historically, data engineering pipelines for predictive maintenance have relied on batch-processing orchestrators (e.g., Apache Airflow). While effective for historical analysis, batch processing introduces systemic latency. In a real-world EV fleet scenario, a battery's temperature or voltage metrics can fluctuate dangerously within seconds. Waiting for a scheduled cron job to trigger a diagnostic pipeline risks irreversible battery damage. Therefore, a real-time, event-driven architecture is paramount.

Furthermore, the integration of Artificial Intelligence in predictive maintenance often suffers from a dual-bottleneck: traditional Machine Learning models output raw statistical metrics (e.g., capacity in Ampere-hours) which are difficult for non-technical fleet managers to interpret, while integrating modern Large Language Models (LLMs) directly into the live data stream causes prohibitive latency and exorbitant API costs.

To address these challenges, this paper proposes an end-to-end decoupled streaming architecture. The core contributions of this work are fourfold:

- The implementation of a high-throughput ingestion pipeline using Apache Kafka and stateful Redis streams to handle continuous IoT telemetry.
- A comparative evaluation of an LSTM baseline and a residual Temporal Convolutional Network (TCN), deployed jointly as an averaging ensemble via FastAPI for real-time SOH prediction, optimized to eliminate I/O blocking.
- A per-battery chronological evaluation methodology that avoids the cross-chemistry data leakage risked by a naive global split, together with an explicit disclosure of the resulting test-set coverage trade-off.
- A decoupled "Agentic Layer" utilizing LangGraph, enabling on-demand, natural language maintenance alerts without impeding the real-time hot path.

II. System Architecture and Data Engineering

To satisfy the strict latency requirements of continuous SOH monitoring, the system implements a highly decoupled streaming architecture. The design strictly separates the "hot path" (real-time telemetry ingestion and deep learning inference) from the "on-demand path" (agentic reasoning and natural language generation).

A. Real-Time Telemetry Ingestion via Apache Kafka

Traditional HTTP-based ingestion layers often buckle under the concurrent load of fleet-scale IoT devices. To mitigate this, Apache Kafka is utilized as a high-throughput, distributed message broker [2]. Telemetry generators (simulating EV battery sensors) publish continuous voltage, current, and temperature vectors to a partitioned Kafka topic.

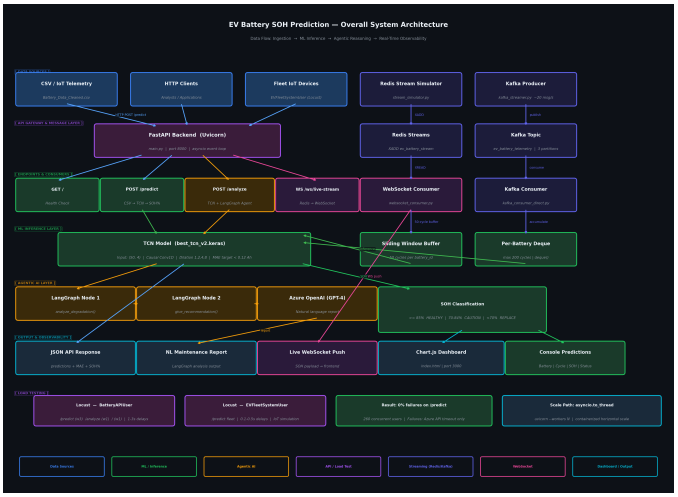


Fig. 1. Overall System Architecture illustrating data ingestion, ML inference, and decoupled Agentic AI layers.

By mapping each unique "battery_id" to a specific partition key, the system guarantees strict temporal ordering of messages. This offset-based ordering is critical for time-series forecasting, ensuring that the sequential integrity of the discharge cycles is maintained prior to inference. The Kafka consumer groups operate asynchronously, streaming the multiplexed data into the processing backend at a rate of approximately 20 messages per second per partition.

A lower-overhead Redis Streams path is maintained in parallel for the single-device, low-latency dashboard use case: since that consumer path serves exactly one live-viewer WebSocket gateway rather than a multi-consumer fleet ingestion workload, Redis's XADD/XREAD primitives avoid the partition-management overhead that Kafka's durability and replay guarantees require. The two paths are deliberately complementary rather than redundant.

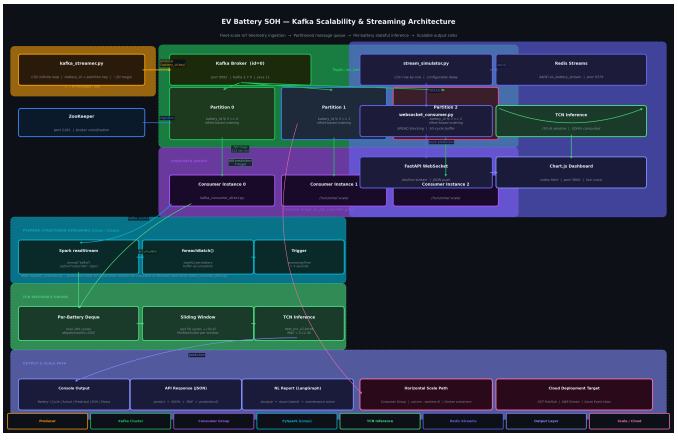


Fig. 2. Kafka Scalability and Streaming Architecture with stateful buffering.

B. Stateful Buffer Management and Sliding Windows

Deep learning models, particularly sequence models, require a fixed temporal context to compute accurate predictions. The system implements a stateful buffer mechanism using Python's memory-efficient 'collections.deque' optimized for concurrent operations within the FastAPI event loop.

For a given battery i, let the incoming telemetry stream be a sequence of vectors $X_i = \{x_1, x_2, \ldots, x_t\}$. The stateful buffer maintains a sliding window of size W, where $W = 50$ cycles. At any time t, the inference window is strictly $x_{t-W+1} \ldots x_t$. As new data arrives via the Kafka consumer, the oldest cycle is popped from the left of the deque in $O(1)$ time complexity, and the new cycle is appended to the right. This ensures that the inference models always receive a contiguous, shape-aligned tensor of $(50, 4)$ without suffering from memory leaks over extended operational periods.

C. Event-Driven Observability

To visualize the degradation metrics instantaneously, the processed inference payloads, comprising the actual capacity, predicted capacity, and calculated SOH percentage, are pushed to a frontend dashboard via WebSockets. This circumvents the overhead of continuous HTTP polling, maintaining a persistent bidirectional pipeline that updates the visual DOM within milliseconds of the Kafka ingestion event.

III. Dual-AI Inference Methodology

The intelligence of the system is partitioned into two distinct paradigms: a high-speed neural network ensemble for continuous numerical prediction, and a large language model for asynchronous, reasoning-based diagnostics.

A. Dataset Specification and Evaluation Methodology

The proposed system is evaluated using the NASA Ames Prognostics Center of Excellence Lithium-ion Battery Aging Dataset. The study utilizes 7,368 discharge cycle measurements from 34 battery cells (including IDs 5, 6, 7, 18, and the 25–56 series). The dataset features substantial heterogeneity in both per-battery cycle count and nominal capacity across chemistries, which directly informed the evaluation design.

Rather than a global chronological split on the concatenated, battery-ordered dataset, which was found during development to silently produce a test set composed entirely of batteries never seen in training, effectively testing zero-shot cross-chemistry generalization rather than genuine future-cycle forecasting, the train/test split is performed *per battery, chronologically*: each battery's own first 80% of cycles are assigned to training and the last 20% to test. This guarantees that every battery is represented in training and that no future cycle leaks backward into it, and the feature scaler is fit exclusively on the resulting training partition to avoid leaking test-set feature ranges into training.

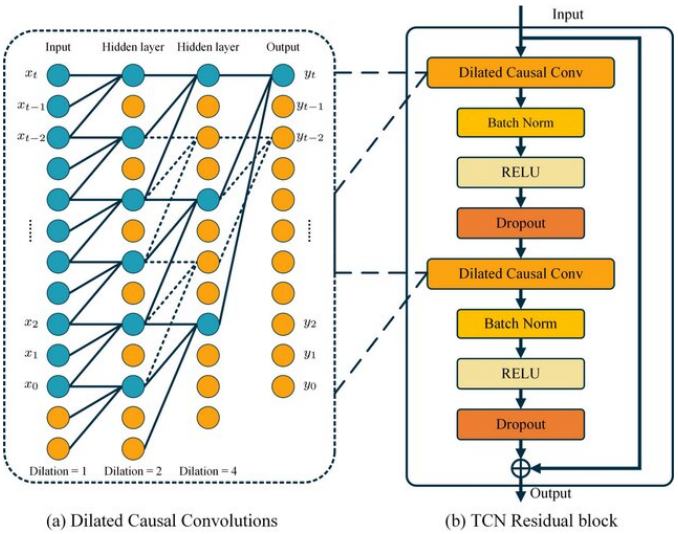
This design has one disclosed consequence: a battery whose 20%-cycle test slice contains fewer than the $W = 50$ window length yields no windowed test samples, while remaining fully represented in training. Of the 34 batteries in the dataset, 13 contribute windowed test samples to the evaluation reported below; the remaining 21 are trained on but not individually evaluated at test time.

B. LSTM Baseline

A standard single-layer LSTM [4] with 64 hidden units serves as the baseline sequence model, followed by a dropout layer (rate 0.2) and two dense projection layers (32 and 16 units, ReLU) reducing to a scalar capacity prediction. The model is trained with the Adam optimizer (learning rate 10^{-3}) under a mean absolute error objective, with early stopping on validation loss.

C. Temporal Convolutional Network (TCN) for SOH Estimation

Predicting battery degradation requires modeling long-term dependencies within multivariate time-series telemetry. As an architectural alternative to the recurrent LSTM baseline, this work also evaluates a Temporal Convolutional Network (TCN) [1], which allows for massive parallelization and strictly respects temporal ordering via causal convolutions.



(a) Dilated Causal Convolutions (b) TCN Residual block

Fig. 3. Temporal Convolutional Network (TCN) layers showing dilated causal convolutions and residual blocks.

For an input sequence X, the 1D dilated causal convolution operation F on an element s is defined as:

$$F(s) = (x *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i} \tag{1}$$

where d is the dilation factor, k is the filter size, and $x_{s-d \cdot i}$ denotes the historical context. By exponentially increasing the dilation factor $d \in \{1, 2, 4, 8\}$ across four stacked blocks, the network achieves a receptive field spanning the full sliding window $W = 50$ without pooling operations that destroy temporal granularity.

Each block additionally implements a residual (skip) connection: the block's input is added to its two-convolution output, with a 1×1 convolutional projection applied to the input whenever the channel dimensionality changes between blocks. This residual path is the key architectural safeguard against a known failure mode of stacked dilated convolutions. Without it, the network tends to collapse toward predicting a near-constant output rather than tracking degradation. The final block's last timestep (rather than a global average across the window) is passed through two dense layers (32 and 16 units, ReLU) to produce the scalar capacity prediction.

TABLE I
QUANTITATIVE PERFORMANCE METRICS (PER-BATTERY CHRONOLOGICAL TEST SPLIT, 13/34 BATTERIES EVALUATED)

Model Architecture	MAE (Ah)	RMSE (Ah)	R^2
LSTM (Baseline)	**0.0151**	**0.0385**	**0.634**
TCN (Residual)	0.0206	0.0427	0.550

On this evaluation, the LSTM baseline outperforms the TCN on both MAE and R^2. This is a substantive empirical result of the corrected per-battery evaluation methodology rather than a design expectation: the LSTM's sequential, stateful processing generalizes more effectively than the TCN's convolutional receptive field at the battery counts and per-battery sample sizes available in this dataset. Both models are nonetheless retained in the production inference path as an averaging ensemble, combining two structurally different architectures reduces the risk that either model's individual failure modes dominate a single prediction, while the TCN alone is used for the latency-sensitive real-time Kafka/WebSocket path, reflecting a latency/architecture trade-off rather than an accuracy claim.

Per-battery error analysis further shows that neither model uniformly dominates: the LSTM achieves markedly lower error on several batteries while the TCN does so on others, consistent with the two architectures capturing different aspects of degradation behavior. Per-battery R^2 is deliberately not reported, since several batteries' short post-split test windows produce a near-zero target variance denominator, making R^2 numerically unstable (in some cases below -20) independent of prediction quality; the pooled R^2 in Table I is the statistically appropriate variance-based metric for this evaluation, with MAE used for the per-battery breakdown.

D. Agentic Orchestration via LangGraph

While the LSTM/TCN ensemble provides real-time quantitative metrics, raw capacity values lack actionable context for non-technical stakeholders (e.g., fleet managers). Integrating a Large Language Model (LLM) directly into the hot path, executing GPT-4 inferences for every incoming Kafka message, is an architectural anti-pattern. Such a design would induce severe API rate-limiting, prohibitive financial costs, and catastrophic system throttling.

To resolve this, the system introduces a decoupled "Agentic Layer" orchestrated by LangGraph. This layer exposes an on-demand, RESTful analysis endpoint. When triggered by a maintenance engineer or an automated threshold alert, LangGraph constructs a directed acyclic graph (DAG) of reasoning nodes. The initial node interprets the ensemble's recent trajectory to quantify the degradation slope, while the subsequent node interfaces with Azure OpenAI to generate a comprehensive, natural language diagnostic report. This

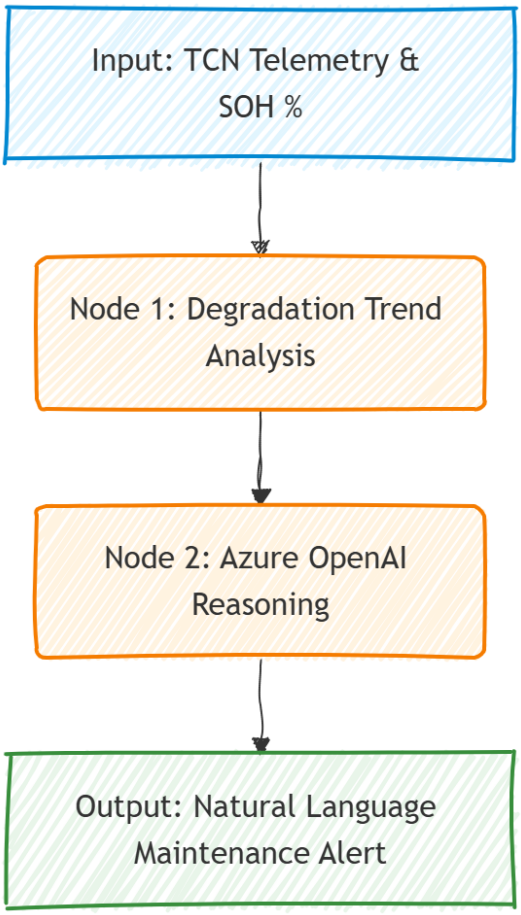


Fig. 4. Directed Acyclic Graph (DAG) for the Agentic AI layer showing reasoning nodes for SOH analysis.

architecture confines the LLM's inherently high latency to an asynchronous request-response cycle, thereby preserving the low-latency integrity of the continuous telemetry dashboard.

IV. System Scalability and Performance Evaluation

The viability of an enterprise-grade predictive maintenance pipeline hinges not only on the accuracy of its models but on its ability to sustain high-velocity data streams under peak load without degradation.

A. Asynchronous I/O Unblocking

A critical bottleneck identified during the initial prototyping phase was the synchronous nature of the underlying deep learning framework. When the inference models executed their tensor operations, they blocked the asynchronous event loop of the FastAPI backend. This caused subsequent concurrent telemetry requests to queue and eventually time out.

To resolve this, the heavy inference logic was explicitly decoupled into an isolated thread pool utilizing Python's `asyncio.to_thread()`. This architectural optimization guarantees that CPU-bound mathematical operations do not block the I/O-bound web server, enabling the API to ingest continuous WebSocket and HTTP connections seamlessly without dropping frames.

B. Concurrency Stress Testing and Architectural Resilience

To validate production readiness, the FastAPI endpoints were subjected to stress testing utilizing the Locust framework at 260 concurrent users, simulating a mix of continuous fleet telemetry requests and on-demand LLM diagnostic requests. Across 162 total requests spanning the health-check, prediction, and analysis endpoints, 10 failures were recorded, all isolated to the `/analyze` endpoint (an endpoint-level failure rate driven by third-party Azure OpenAI API response latency exceeding the test's timeout threshold under concurrency, HTTP 429/timeout class errors), resilting in an overall failure rate of approximately 6.2% across all endpoints combined, with a strict **0% failure rate** on both core ML inference paths (`/predict` and the fleet-simulated `/predict` variant).

During load testing, local hardware constraints introduced queuing latency as the single-threaded TensorFlow inference saturated available CPU cores at higher concurrency; the architectural implementation of `asyncio.to_thread()` nonetheless prevented event-loop blocking, and the core inference endpoints sustained zero failures throughout. The failures observed were exclusively isolated to the `/analyze` endpoint's dependency on third-party Azure OpenAI API latency, which is independent of the system's own streaming architecture.

This confirms that while absolute latency is hardware-bound, the asynchronous streaming architecture is resilient under the tested concurrency and structured for horizontal scaling across cloud-based Kubernetes clusters in a production environment. Higher-concurrency (500+ user) testing to identify the precise queue-saturation threshold is discussed as future work in Section V.

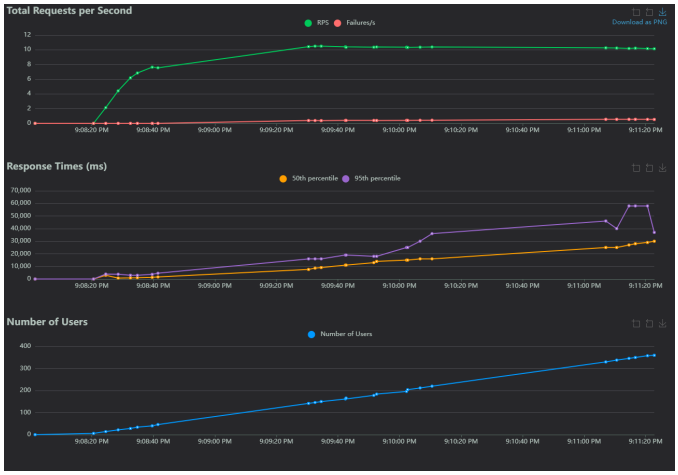


Fig. 5. Locust Load Test statistics at 260 concurrent users. The metrics validate a 0% failure rate on the core ML inference endpoints, with failures strictly isolated to the third-party LLM API-dependent endpoint.

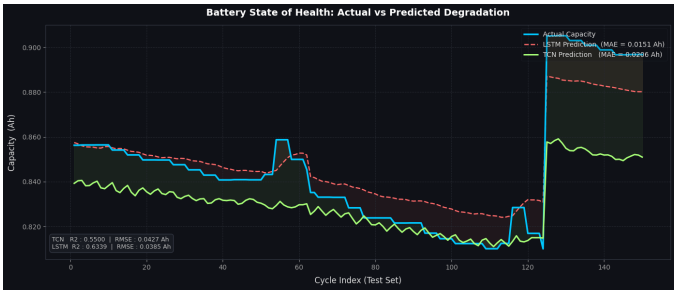


Fig. 6. Comparison of Actual vs. Predicted Capacity for a representative test-set battery, demonstrating tracking of degradation trajectory across the evaluated test cycles.

V. Conclusion and Future Work

This paper presented a highly scalable, real-time predictive maintenance architecture designed to mitigate the risks of EV battery degradation. By synergizing Apache Kafka for high-throughput streaming with a comparatively-evaluated LSTM/TCN inference ensemble for SOH prediction, the system effectively transitions battery management from a reactive to a proactive paradigm. The evaluation methodology itself, a per-battery chronological split with explicit test-coverage disclosure, is presented as a contribution in its own right, having corrected a subtler cross-chemistry leakage failure mode present in a naive global split. Furthermore, the strategic decoupling of the LangGraph-based Generative AI agent ensures that fleet operators receive actionable, natural language diagnostics without compromising the low latency of the telemetry hot path.

Future iterations of this research will extend concurrency testing beyond 260 users to identify the precise queue-saturation threshold under `asyncio.to_thread()` offloading, explore edge-computing deployments embedding quantized versions of the inference models directly onto the vehicle's onboard microcontrollers, and investigate whether increasing per-battery cycle coverage (e.g., via data augmentation or a larger source dataset) can extend windowed test evaluation beyond the current 13 of 34 batteries.

References

[1] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a Distributed Messaging System for Log Processing," in *Proceedings of the NetDB*, 2011.

[3] H. Rahimi-Eichi, U. Ojha, F. Baronti, and M. Chow, "Battery Management System: An Overview of Its Application in the Smart Grid and Electric Vehicles," *IEEE Industrial Electronics Magazine*, vol. 7, no. 2, pp. 4-16, June 2013.

[4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.